

PAC Learning and Philosophy

Kyle Macadaeg

15 May 2026

1 Introduction

While we tend to view PAC learning under the lens of machine learning, it is, like other mathematical definitions, a general enough concept that it could be applied to other contexts. Perhaps unsurprisingly, we could try to apply it in the context of ordinary learning. Specifically, we'll look at how PAC learning can be adapted to subjects ostensibly unrelated to machine learning, namely the epistemology of science and the process of acquiring language for the first time. To motivate the generality of the PAC learning definition, let's try to intuitively explain three key aspects:

1. **The δ parameter.** With probability less than or equal to δ , our algorithm will produce a strong hypothesis. This captures one of the imperfections of the learning process: it is entirely possible that we might fail. However, the fact that δ can be made arbitrarily small suggests that we should be able to mitigate this issue. We allow ourselves the possibility of making mistakes, but we must be able to combat mistakes by taking more data.
2. **The ϵ parameter.** A hypothesis is considered strong when the probability of error is less than or equal to ϵ . Like the δ parameter, this captures another imperfection of the learning process: that even when things go right, we may not have a perfect solution. Again like the δ parameter, while we allow imperfections, we must be able to mitigate them with time.
3. **Polynomial runtime.** Finally, we would like to restrict our attention to efficient PAC learning, i.e. that which takes polynomial time. Naturally, this leads to the reasonable question of why we should equate "polynomial" and "efficient". We're restricting ourselves to thinking only about worst-case performance asymptotically. Partially justifying this, empirically, we rarely see "efficient" exponential algorithms (something looking like $O(1.0001^n)$) nor "inefficient" polynomial algorithms (like $O(n^{10001})$). If we allow ourselves to accept "polynomial" as a definition for "efficient", then polynomial time condition captures the necessity of economy. We can't allow ourselves to draw arbitrary amounts of data; we have to take a relatively small amount of data and draw conclusions from that.

In total, while the PAC learning definition may seem ad hoc, contrived, or arbitrary, each part of its definition reflects a reality of ordinary learning.

2 Science and Epistemology

We will first examine epistemology, the philosophy of knowledge, as applied to science. While science and math are intertwined, the epistemology of mathematics cannot be applied to science. In mathematics, we start by taking certain statements to be true and reason about their consequences. We allow ourselves the certainty of the consequences given the starting points, but are agnostic to whatever truth the starting points may hold. While the mathematician concerns themselves with multiple structures, the scientist only ever concerns themselves with the singular universe that they find themselves in. As such, they can no longer afford indifference to the truth of their starting points, but they also have no way to ascertain any starting points in the first place!

The scientist is thus forced to try to investigate the universe without knowing the fundamental rules that govern it. This process is crystallized into the scientific method of observation, hypothesizing, and experimentation. The "hypothesizing" aspect already calls to mind the PAC learning setting; the goal of a learning algorithm is to produce a hypothesis concept. A little less obviously, we can draw parallels between the other parts of the scientific method with other parts of PAC learning. Treating the universe's laws as a concept to learn, observation can be likened to drawing samples from an example oracle while experimentation can be likened to drawing samples from an MQ oracle.

These parallels are not surprising; both science and the PAC learning setting are built upon inductivism, where we use past data to reason about the future. An example would be to observe 500 black ravens, then concluding that the next raven will also be black. Much of the philosophy surrounding inductivism can be rederived from the PAC learning view; as a result, we can think of PAC learning not as an application of inductivism to machine learning but as a formalization of inductivism. With this formalization, we can also address new points regarding inductivism.

2.1 Rederiving Popperian Philosophy

First, let's address some challenges to inductivism; for instance, why should we even trust inductivism? Historically, the scientific method hasn't been completely successful. Seeing that waves travel through media, it is inductively reasonable to assume that light must also travel through such a medium, but the theory of the luminiferous aether was ruled out by the Michelson-Morley experiment. Even among the successes of science, we have nothing perfect. Newtonian mechanics was undermined by relativity and quantum mechanics. To this day, we do not have a theory of everything that reconciles the latter two.

The modern view on science is that of Karl Popper, that the scientific never truly produces knowledge. It simply produces statements that are falsifiable, but have yet to be falsified. At no point do we ever claim to have certainty. Similarly, in PAC, we never claim to have perfect knowledge of a target concept by the existence of the δ and ϵ parameters. We can make an analogue between the production of poor scientific hypotheses to the probability to create poor hypotheses in the PAC setting afforded to us by the δ parameter. We may not always produce good hypotheses, but science undoubtedly creates many good ones. Furthermore, the failure of successful theories to be truly comprehensive can be likened to the probability that good hypotheses err in the PAC setting afforded to us by the ϵ parameter. We could even go a step further, and say that the failures of Newtonian mechanics to explain relativity or quantum mechanics was because the observations that led to Newtonian mechanics are due to only having observations in the macroscopic scale, i.e. because we only ever sampled from a specific distribution. Thus, we see that PAC agrees with modern scientific philosophy.

2.2 Explaining Occam's Razor, Computationally

Despite the view that inductivism does not ever truly produce knowledge, it is still undeniable that it has been largely successful in producing explanations. Attempts to explain this generally appeal to Occam's Razor, that simpler hypotheses tend to be preferable. Of course, this raises a few questions of its own.

1. **What do we mean by simpler?** The clear answer in the PAC setting is the size of the concept (e.g. Kolmogorov complexity). This is suitable when talking about mathematical objects, but it is still unclear how to adapt this to the real-world setting of science. For instance, "green" and "blue" may seem more "primitive" in comparison to the concepts "green until January 1st, 2030 and blue afterwards" and "blue until January 1st, 2030 and green afterwards". However, define the former as "grue" and the latter as "bleen", then we can use these as primitive concepts to define "green" as "grue until January 1st, 2030 and bleen afterwards" and "blue" as "bleen until January 1st, 2030 and grue afterwards". Why then should we think that "green" and "blue" are the simpler ideas in the first place? This is Nelson Goodman's "new riddle of induction", which we cannot entirely address. However, theoretical computer science can offer the insight that perhaps trying to designate one as simpler than the other here is unimportant; whichever you choose as the "primitive" concepts, simply defining the others in terms of your chosen concepts takes only constant overhead, and is thus insignificant.
2. **Why should simpler hypotheses be preferred?** Are simpler ideas just simpler to work with, and thus easier for us humans? Is there some truth to simpler hypotheses just being more effective? Do we just like simpler hypotheses? Quine posits that it's a bias of our perception and

the way that science is performed. Here, computational learning theory can provide some insight, as Occam's Razor has its own version in computational learning theory. We say that there is a learner is an Occam learner if, given a sample, it can produce a hypothesis (of restricted size) consistent with that sample. It is efficient if this can be done in polynomial time. The computational learning theory version of Occam's Razor states that an (efficient) Occam learner is also a (efficient) PAC one. This theorem lends credence to the process of inductivism; just as we treat PAC learning as a formalization of inductivism, we can treat the computational learning theory version of Occam's Razor as a formalization of Occam's Razor.

In total, we have seen that PAC learning doesn't just agree with the current philosophy, but can partially address some of the outstanding questions.

3 Language Acquisition and Psychology

Linguist Noam Chomsky took an interest in linguistics not for its own sake, but to draw insights into the human mind. During his life, the dominant belief of linguistics was behaviorism, that behavior is governed by stimuli and responses (i.e. reinforcement or punishment). Chomsky himself argued that the behaviorist concepts of stimuli and responses were not applicable to language learning. First, he made a distinction between "performance", the ways that a person uses language, and "competence", the things that a person knows about a language. Notably, there are infinitely many sentences, and thus performance cannot possibly be comprehensive, yet a person can certainly recognize by their competence grammatically correct sentences. How can we analyze language learning then through a behaviorist lens, if there are infinitely many stimuli?

A person will only ever receive finitely many sentences, and yet, of the infinitely many languages that could fit those sentences, all children end up learning roughly the same language. Chomsky argues that there must be some sort of linguistic bias called the "Universal Grammar", that predisposes them to certain ways of thinking about language.

At this point, we can start to see the parallels to PAC learning. Here, a concept is a language, and the samples from that concept are grammatically well-formed sentences. The key elements of PAC learning have clear analogues here. The δ parameter would capture those who struggle with learning language and the ϵ parameter would capture the mistakes that even the comfortable native speaker is bound to make. The efficiency requirement is also important here; children pick up languages fast, with only few sentences (comparatively to the infinitely many) to guide their learning.

Still, to formalize language learning in this way means we make a few assumptions.

1. **The input data has no noise**, i.e. a child will only ever receive correctly formed sentences of their language. This is needed since the example oracle

in the PAC setting makes no mistakes. While not reflective of reality, this is a fair enough assumption to make; a child will generally receive mostly correct sentences in the process of learning languages.

2. **The way in which all children learn language can be represented by a singular algorithm.** This is another assumption that is not reflective of reality, but a necessary assumption for us to formulate our formalization. Whether the mind can even at all be represented by a computer is outside of our scope.
3. **PAC learning is the correct setting for this problem.** Perhaps the fact that δ or ϵ can be chosen arbitrarily is too strong of a requirement. It would suggest that any child could learn a language if we simply throw enough sentences its way, and that with enough sentences, it will never make any mistakes within a reasonable time frame.

With this, we can return to Chomsky. Part of our formalization will involve his contributions to grammar. Specifically, we will look at the Chomsky hierarchy of languages. Chomsky designated four types of languages, numbered 0 through 3. The lowered numbered types have more complex grammars, and thus contain all of the languages of the higher-numbered types.

Type	Name	Associated Automaton
0	Recursively Enumerable	Turing Machine
1	Context-sensitive	Linear-bounded Non-deterministic Turing Machine
2	Context-free	Pushdown Automaton
3	Regular	Deterministic Finite Automaton

Our goal is to argue that languages are not PAC learnable. In doing so, that would suggest that we would need more information than just type to learn languages, supporting Chomsky's theory of the Universal Grammar.

Linguists believe that natural languages are either of Type 1 or Type 2. The natural question is then to check if these are PAC learnable. To answer this, let's first look at Type 3. Languages are just finite subsets of all words that can be formed from the alphabet, and finite subsets are not PAC learnable. Even if we restrict our languages to have words of bounded length, there are still too many concepts and the VC dimension grows superpolynomially with this length. Since all Type 3 languages are Type 1 and Type 2, this would suggest that natural languages are not PAC learnable.

Of course, the traditional PAC setting doesn't really reflect the process by which we learn language. Language is a two-way process; a child will also speak and be corrected. This would be akin to a membership query, where we can check whether a sentence is correctly formed. However, even with membership queries, we would only be able to learn regular languages effectively (as we can learn deterministic finite automata, which recognize regular languages), and not Type 1 or Type 2 languages. In fact, Type 1 languages are known not to be PAC+MQ learnable unless $P=NP$.

If we reformulate our problem to have a target *grammar* rather than a target *language* (as multiple grammars can generate the same language), then we can learn a strict superset of the regular languages, but a strict subset of the context-free languages, where our membership queries also allow us to ask about productions from non-starting symbols. This would be akin to a child asking whether a certain phrase is a well-formed prepositional phrase, which is unrealistic for learning a first language. Even still, this would only be a subset of the Type 2 languages, so we still cannot get to Type 1 languages.

In total, it would seem that natural languages are not PAC learnable even with membership queries, suggesting that more knowledge beyond simply type is necessary to learn languages. This would lend credence to Chomsky's idea of the Universal Grammar.

4 Conclusion

As a final note, it is worth noting that while we have discussed learning in the traditional sense, PAC learning could be applied to other contexts related to improvement. Namely, we could speak of a species "learning" to adapt in a certain environment. This leads us to the work of Leslie Valiant, the creator of PAC learning, who formulated a "theory of evolvability" under the PAC framework.

PAC learning is a much more general concept than just a nice requirement for a binary classification algorithm in machine learning. The same complexities that let it represent aspects of ordinary learning well provide us with a strong foundation for any context where we have improvement, but never perfection.

5 Bibliography

- S. Aaronson. Why Philosophers Should Care About Computational Complexity
<https://www.scottaaronson.com/papers/philos.pdf>
- R. de Wolf. Philosophical Applications of Computational Learning Theory:
Chomskyan Innateness and Occam's Razor
<https://homepages.cwi.nl/~rdewolf/publ/philosophy/phthesis.pdf>